Aim: Implementation of the following Page Replacement Algorithms (a)FCFS (b) LRU (c) OPTIMAL

A. FCFS (First Come First Serve)

Algorithm:

1. **Start**.
2. Initialise the number of page frames as empty
3. Read the page reference string on by one and check whether reach page is present in the frame
4. If the page is absent, replace the oldest page using FIFO rule and increment the page fault count.
5. Display the allocation status and allocated block numbers.
6. **Stop**.

B. LRU (Least Recently Used)

Algorithm:

7. **Start**.
8. Initialise the number of page frames as empty
9. Read the page reference string on by one and check whether reach page is present in the frame
10. If the page is absent, replace the Least Recently used page and increment the page fault count.
11. If present, update its recent usage and increment the page hit count.
12. Display the index block and its allocated block mapping.
13. **Stop**.

C. OPTIMAL

Algorithm:

14. **Start**.
15. Read the page reference string on by one and check whether reach page is present in the frame.
16. If the page is absent, replace the page that will not be used for the longest time in the future.
17. Increment the page fault count. If present, increment the Page Hit count.
18. **Stop**.

A. FCFS

Program:

```
#include <stdio.h>
printf(”26075A6204\n”);
int main() {int n, f, p[20], a[10], i, j, k = 0, faults = 0, hit;
   printf("Enter no of pages and frames: ");
   scanf("%d%d", &n, &f);
   printf("Enter page reference string: ");
   for(i = 0; i < n; i++)
      scanf("%d", &p[i]);
   for(i = 0; i < f; i++) a[i] = -1;
   for(i = 0; i < n; i++) {
      hit = 0;
      for(j = 0; j < f; j++)
         if(a[j] == p[i]) hit = 1;
      if(!hit) {
         a[k] = p[i];
         k = (k + 1) % f;
         faults++;
      }}
   printf("Page Faults = %d", faults);
   return 0;}
```

Testcases:

```
Roll no: 26075A6204
Enter no of pages and frames: 20 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Page Faults = 15
```

```
Roll no: 26075A6204
Enter no of pages and frames: 18 3
Enter page reference string: 1 2 3 2 1 4 2 3 1 4 3 2 1 4 1 2 3 4
Page Faults = 8
```

```
Roll no: 26075A6204
Enter no of pages and frames: 21 3
Enter page reference string: 5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
Page Faults = 11
```

B. LRU

Program:

```
#include <stdio.h>
int main() {
   printf(”26075A6204\n”);
   int n, f, p[20], a[10], t[10], i, j, k, faults = 0, hit, time = 0;
   printf("Enter no of pages and frames: ");
   scanf("%d%d", &n, &f);
   printf("Enter page reference string: ");
   for(i = 0; i < n; i++) scanf("%d", &p[i]);
   for(i = 0; i < f; i++) a[i] = -1;
   for(i = 0; i < n; i++) {
      hit = 0;
      for(j = 0; j < f; j++)
         if(a[j] == p[i]) {
            hit = 1;
            t[j] = ++time;}
      if(!hit) {
         k = 0;
         for(j = 1; j < f; j++)
            if(a[j] == -1 || (a[k] != -1 && t[j] < t[k])) k = j;
         a[k] = p[i];
         t[k] = ++time;
         faults++;}}
   printf("Page Faults = %d", faults);
   return 0;}
```

Testcases:

```
26075A6204
Enter no of pages and frames: 20 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Page Faults = 12
```

```
26075A6204
Enter no of pages and frames: 18 3
Enter page reference string: 1 2 3 2 1 4 2 3 1 4 3 2 1 4 1 2 3 4
Page Faults = 12
```

```
26075A6204
Enter no of pages and frames: 13 3
Enter page reference string: 1 2 3 4 2 1 5 6 2 1 2 3 7
Page Faults = 11
```

C. OPTIMAL

Program:

```
#include <stdio.h>
int main() {
   printf(”26075A6204\n”);
   int n, f, p[20], a[10], i, j, k, faults = 0, hit, far, pos;
   printf("Enter no of pages and frames: ");
   scanf("%d%d", &n, &f);
   printf("Enter page reference string: ");
   for(i = 0; i < n; i++) scanf("%d", &p[i]);
   for(i = 0; i < f; i++) a[i] = -1;
   for(i = 0; i < n; i++) {
      hit = 0;
      for(j = 0; j < f; j++)
         if(a[j] == p[i]) hit = 1;
      if(!hit) {pos = 0;
         far = -1;
         for(j = 0; j < f; j++) {
            for(k = i + 1; k < n && a[j] != p[k]; k++);
            if(k > far) {far = k;
               pos = j;}}
         a[pos] = p[i];
         faults++;}}
   printf("Page Faults = %d", faults);
   return 0;}
```

Testcases:

```
26075A6204
Enter no of pages and frames: 20 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Page Faults = 12
```

```
26075A6204
Enter no of pages and frames: 18 3
Enter page reference string: 1 2 3 2 1 4 2 3 1 4 3 2 1 4 1 2 3 4
Page Faults = 12
```

```
26075A6204
Enter no of pages and frames: 13 3
Enter page reference string: 1 2 3 4 2 1 5 6 2 1 2 3 7
Page Faults = 11
```